

Last modified on

Sports Data Soccer API - Match Playtime Feed (MA43)

Get information about playtime and net-playtime of a match.

Overview

The Soccer **Match Playtime** feed provides information about playtime, net-playtime, as well as definition of "dead" periods e.g., how long is the ball out of play after a goal, before a free-kick, before a throw-in, whilst there is an injury break.

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our **FAQs**. We also recommend that you take a look at our **API Overview and Authentication Guide**. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Contact your Stats Perform representative for a full list of available comps.

UPDATED	POLLING FREQUENCY		
	TIER 15+13	TIER 14+12+10	ALL OTHER TIERS
Every two minutes whilst the match is in-flight and at every start/stop of the period	Once every two minutes	Once every 8 hours for the 7-day period after the match has ended	N/A

Match Playtime (MA43) Guide

USAGE	FEED
GET match playtime list by fixture UUID inline URL path	/soccerdata/matchplaytime/{outletAuthKey}/{fixtureUuid}?q
GET match playtime list by fixture UUID	/soccerdata/matchplaytime/{outletAuthKey}?{fx=fixtureUuid}&
GET match playtime list by tournament calendar UUID	/soccerdata/matchplaytime/{outletAuthKey}?{tmcl=tmclUuid}&

If your request URL is as follows (does not contain a mandatory fixture UUID), the feed returns an error:
/soccerdata/matchplaytime/{outletAuthKey}

Query Parameters And Example Request Parameters

This feed also makes use of the **Global query parameters**, and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you must query the `/matchPlayTime` feed by a fixture (match) UUID.

Legacy IDs: You can use the `fx` query parameter to filter by an Opta legacy ID (only one ID is supported - you cannot specify a comma-separated list of IDs).

How to use query parameters and make successful requests:

In our guides, we use HTTPS syntax for our example `GET` requests (rather than `cURL`, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You **MUST** format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the **main guide** and **API Overview and Authorisation guide**.

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` this must be after the Outlet Authentication Key (and endpoint, if applicable). for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format), entity query parameter and UUID.
- Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or `-` (multiple values, 'OR') or `!` ('NOT').
- Use the required `_` (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces `{ }` from your calls (we include these as an example only).
- If using the JSONP format (`_fmt=jsonp`) you **MUST** use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Parameter	Value and description
fx	GET Match Playtime list by fixture UUID inline URL path. GET https://api.performfeeds.com/soccerdata/matchplaytime/{outletAuthKey}/{fixtureUuid}?_fmt=xml&_rt=b
fixtureUuid	GET Match Playtime list for a match by the fixture UUID . GET https://api.performfeeds.com/soccerdata/matchplaytime/{outletAuthKey}?fx=fixtureUUID&_fmt=xml&_rt=b
tmcl	GET Match Playtime list by tournament calendar UUID .

Parameter	Value and description
	GET https://api.performfeeds.com/soccerdata/matchplaytime/{outletAuthKey}?{tmcl=tmclUUID}&_fmt=xml&_rt=b

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- [XML](#)
- [JSON](#)

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

▶<matchPlayTime> (XML only)

▶<matchInfo>

▶<description>

▶<sport>

▶<ruleset>

▶<competition>

▶<country>

▶<tournamentCalendar>

▶<stage>

▶<contestants> (XML only)

▶<contestant>

▶<country>

▶<venue>

▶<liveData>

▶<matchDetails>

▶<periods> (XML only)

▶<period>

▶<suspensions>

▶<suspension>

▶<scores>

▶<ht>

▶<ft>

▶<et>

▶<pen>

▶<total>

▶<playTime>

▶<firstHalf>

▶<stat>

▶<secondHalf>

▶<stat>

▶<firstHalfExtra>

▶<stat>

▶<secondHalfExtra>

▶<stat>

▶<total>

▶<stat>

▶<contestant>

▶<stat>

▶<playerLineUp>

▶<matchPlayer>

▶<stat>

▶<matchDetailsExtra>

▶<matchOfficials>

▶<matchOfficial>

List Of Stats

▶Stats - names and descriptions

